PYTHON OPERATORS

Complete classroom notes with explanations, code examples and practice

Learning objective: Understand how Python operators work, how to use them in expressions, and how Python decides the order in which operations are performed.

1. What are operators?

An **operator** is a symbol or keyword that tells Python to perform an operation on one or more values. The values used with an operator are called **operands**.

```
total = 10 + 5
# 10 and 5 are operands; + is the operator.
# total becomes 15.
```

Python has several groups of operators. This lesson covers arithmetic, assignment, ternary, comparison, logical, identity, membership and bitwise operators, followed by operator precedence.

2. Arithmetic operators

Arithmetic operators perform mathematical calculations such as addition, subtraction, multiplication and division.

Operator	Operation	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3.333...
//	Floor division	10 // 3	3
%	Modulus (remainder)	10 % 3	1
**	Exponentiation (power)	2 ** 3	8

Example: arithmetic in a program

```
a = 17
b = 5

print(a + b)   # 22
print(a - b)   # 12
print(a * b)   # 85
print(a / b)   # 3.4
print(a // b)  # 3
print(a % b)   # 2
print(a ** b)  # 1419857
```

Remember: `/` gives a division result as a float. `//` performs floor division (rounds the result down to the next whole-number value). `%` gives the remainder.

3. Assignment operators

The assignment operator = stores a value in a variable. Compound assignment operators perform an operation and assign the updated result back to the same variable.

Operator	Example	Equivalent idea
=	x = 10	Store 10 in x
+=	x += 3	x = x + 3

Operator	Example	Equivalent idea
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
//=	x //= 3	x = x // 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3
&=, |=, ^=	x &= 3 etc.	Bitwise update
>>=, <<=	x >>= 1 etc.	Bit shift update

```
balance = 100
balance += 50
print(balance)  # 150

balance -= 20
print(balance)  # 130

balance *= 2
print(balance)  # 260
```

The expression `balance += 50` is a shorter way of writing `balance = balance + 50`.

4. Ternary operator (conditional expression)

A conditional expression chooses one of two values based on a condition. It is often called the **ternary operator** because it involves a condition and two possible results.

```
age = 20
message = "Adult" if age >= 18 else "Minor"
print(message)  # Adult
```

Read it as: use `'Adult'` if the age is at least 18; otherwise use `'Minor'`. For complex decisions, a normal `if/else` statement is usually easier to read.

5. Comparison operators

Comparison operators compare two values. Every comparison produces a Boolean result: `True` or `False`.

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 2	True
>	Greater than	8 > 4	True
<	Less than	2 < 1	False
>=	Greater than or equal to	6 >= 6	True
<=	Less than or equal to	3 <= 2	False

```
marks = 76

print(marks >= 40)  # True: passed
print(marks == 100) # False: not a perfect score
```

Common confusion: = assigns a value, while == checks whether two values are equal.

6. Logical operators

Logical operators combine conditions or reverse a condition. They are commonly used in `if` statements.

Operator	Meaning	When it returns True
and	Logical AND	Both conditions are True
or	Logical OR	At least one condition is True
not	Logical NOT	The condition is False; it reverses the truth value

```
age = 22
has_id = True

print(age >= 18 and has_id)  # True
print(age < 18 or has_id)    # True
print(not has_id)            # False
```

Python uses short-circuit evaluation: with `and`, it stops once the result is known to be false; with `or`, it stops once the result is known to be true.

7. Identity operators

Identity operators check whether two variables refer to the **same object** in memory. They do not simply compare whether two values look equal.

`is` returns True when both references point to the same object. `is not` returns True when they point to different objects.

```
first = [1, 2, 3]
second = [1, 2, 3]
third = first

print(first == second)  # True: values are equal
print(first is second)  # False: separate list objects
print(first is third)   # True: same object
print(first is not second)  # True
```

Use == for value equality. Use `is` when checking object identity. A common correct use is `if result is None:`.

8. Membership operators

Membership operators check whether an item exists inside a sequence or collection, such as a string, list, tuple, set or dictionary.

```
fruits = ["apple", "banana", "mango"]

print("banana" in fruits)      # True
print("grapes" not in fruits)  # True

word = "Python"
print("Py" in word)            # True
```

For a dictionary, `in` checks keys by default, not values:

```
student = {"name": "Asha", "age": 21}

print("name" in student)       # True
print("Asha" in student)       # False
print("Asha" in student.values())  # True
```

9. Bitwise operators

Bitwise operators work on the binary (0 and 1) representation of integers. They are useful in areas such as permissions, flags, low-level programming and certain performance-sensitive tasks.

Operator	Name	What it does
&	AND	Bit is 1 only when both bits are 1

Operator	Name	What it does
|	OR	Bit is 1 when either bit is 1
^	XOR	Bit is 1 when the bits are different
~	NOT	Inverts the integer's bits (Python uses signed integer semantics)
<<	Left shift	Shifts bits to the left
>>	Right shift	Shifts bits to the right

Example using binary values

```
a = 5   # binary: 0101
b = 3   # binary: 0011

print(a & b)   # 1  (0001)
print(a | b)   # 7  (0111)
print(a ^ b)   # 6  (0110)
print(a << 1)  # 10 (1010)
print(a >> 1)  # 2  (0010)
print(~a)      # -6
```

For ~a, Python's result is `-(a + 1)`. Therefore, ~5 is `-6`.

10. Operator precedence

An expression can contain several operators. **Operator precedence** determines which operation Python performs first. Operators with higher precedence are evaluated before operators with lower precedence. Parentheses can be used to make the intended order clear.

Priority (high to low)	Operators / category
1	Parentheses and grouping: ( )
2	Exponentiation: **
3	Unary plus, minus and bitwise NOT: +x, -x, ~x
4	Multiplication, matrix multiplication, division, floor division, modulus: *, @, /, //, %
5	Addition and subtraction: +, -
6	Bit shifts: <<, >>
7	Bitwise AND: &
8	Bitwise XOR: ^
9	Bitwise OR: |
10	Comparisons, membership and identity: <, <=, >, >=, ==, !=, in, not in, is, is not
11	Logical NOT: not
12	Logical AND: and
13 (low)	Logical OR: or
Special	Conditional expression: value_if_true if condition else value_if_false

Example: precedence in action

```
answer = 10 + 2 * 3
print(answer)  # 16: multiplication happens first

answer = (10 + 2) * 3
print(answer)  # 36: parentheses happen first

answer = 2 ** 3 ** 2
print(answer)  # 512: exponentiation groups right to left
```

Even when you know the precedence rules, use parentheses when an expression may be difficult to read. Clear code is more important than writing the shortest expression.

11. Quick revision: which operator should you use?

• Calculate a total: arithmetic operators such as + and `*`.
• Update a variable: assignment operators such as +=.
• Choose between two values: a conditional expression.
• Compare two values: comparison operators such as >= and ==.
• Combine conditions: `and`, `or`, `not`.
• Check whether references point to the same object: `is`.
• Check whether an item is present: `in`.
• Work with integer bits: bitwise operators.

12. Practice questions

Try to predict the output before running the code.

```
# 1. Arithmetic
print(17 // 4)
print(17 % 4)

# 2. Assignment
count = 5
count += 3
print(count)

# 3. Ternary expression
age = 17
print("Adult" if age >= 18 else "Minor")

# 4. Comparison and logical operators
x = 12
print(x > 10 and x < 20)

# 5. Membership
languages = ["Python", "Java"]
print("Python" in languages)

# 6. Identity
a = [1, 2]
b = a
print(a is b)

# 7. Bitwise
print(6 & 3)

# 8. Precedence
print(4 + 3 * 2)
```

Answers with short explanations

1. `4` and `1` — quotient and remainder.
2. `8` — 5 plus 3.
3. `Minor` — 17 is below 18.
4. `True` — both comparisons are true.
5. `True` — Python is in the list.
6. `True` — both variables refer to the same list object.
7. `2` — binary 110 AND 011 gives 010.
8. `10` — multiplication happens before addition.

Key takeaways

Python operators allow programs to calculate values, assign and update variables, compare data, make decisions, test membership and identity, and manipulate integer bits. Learn the purpose of each operator group and use parentheses to keep complex expressions easy to understand.

End of notes